

R3.07 SQL dans un langage de programmation

L'objectif de ce TD est d'écrire un seul **DAOStatement** et un seul **DAOPreparedStatement** pour toutes les tables avec les fonctions **CRUD**.

Les sources sont à déposer sur madoc en fin de la dernière séance.

1- Créer un nouveau projet en **Kotlin** sous **IntelliJ**(Choisir *IntelliJ* et non *Gradle* lors de la création du projet).

2- Créer un **package BD** qui contient la classe **SessionOracle**(voir fichier sur Madoc) qui nous permet à se connecter à la base de données Oracle. Cette classe contient un attribut de type **Connection** plus le **login et mot de passe oracle**

N'oubliez pas de charger les fichiers « jar » qui se trouvent aussi sur madoc. Ajoutez ces fichiers aux librairies de votre projet.

Modules Setting....Dependencies...+...Jars or Directories....(choisir les deux jar et faire apply)

3- Créer un package **Bean**. Dans ce package, vous développez les classes *employe, service, projet, travail et concerne*, qui correspondent aux table sous oracle et contiennent **les mêmes attributs que les tables**.

5- Créer un package **DAO**. Ce dernier package correspond à la couche accès aux **métadonnées**. Nous faisons correspondre une classe **DAOStatement et DAOPreparedStatement** pour toutes les table(voir madoc). Ces classes contiennent des méthodes ou fonctions qui font appels aux différentes requêtes. Elle contient aussi un attribut ou une instance de *sessionOracle*. Chacune des fonctions contiendra le **nom de la table** en paramètre ou une instance d'**objet(obj : Any)**.

La méthode **read** sera appliquée pour n'importe quelle table. *read(tableName : String)*.
La requête SQL est sous la forme :

val requete = "SELECT * FROM \$tableName"

L'accès aux métadonnées ce fait de la manière suivante :

- *ResultSet rs = stmt.executeQuery(requete) ;*
- *ResultSetMetaData rsmd= rs.metaData;// les méta données d'oracle*
- *int nbcolonnes = rsmd.columnCount; // Le nombre de colonnes retourné par le Select*
- *String nomcol = rsmd.getColumnName(i); // Nom de la colonne où i est la position de l'attribut dans le select*
- *val type = rsmd.getColumnType(i)// Type de la colonne*

Avant d'afficher le résultat de la requête, vous affichez les noms et les types des différents attributs.

Vous récupérez les données suivant le type de l'attribut, et vous faites un **getInt** ou un **getString**.

Vous développez les différentes fonctions *Create*, *Delete* et *Update* au fur et à mesure dans les classes *DAOStatement* et *DAOPreparedStatement*.

Dans le cas de la fonction **Create** en Kotlin, la requête SQL est construite en fonction des valeurs récupérées dans l'instance(*obj : Any*).

La fonction *Create* est sous forme :

Dans le *DAOStatement* : **val requete: String="INSERT INTO \$nomtable VALUES(v1,v2,v3,)"**

et dans *DAOPreparedStatement* : **val requete : String=Insert into \$nomTable values(?,?,?,.....)**

Où chaque paramètre est initialisé avec les méthodes *stmt.setInt(position du paramètre, valeur)*.

Les fonctions *Delete* et *Update* récupèrent les clés primaires de la table pour construire la requête SQL.

6- Compléter la fonction main qui permet de tester la connexion à la base de données, de créer des *instances des différentes tables* et d'exécuter les différentes opérations.

Afin de vérifier l'exécution des méthodes **Kotlin**(create, update et delete) sur la BD, vous pouvez encapsuler par un appel à la méthode *read()* **avant et après**.